THE ULTIMATE BITWISE OPERATIONS MASTERCLASS

For Embedded Systems Engineers — From Zero to Hero in C

Welcome, future embedded engineer! 🎉 Bitwise operations are **THE most important skill** in embedded systems. You'll use them every single day — configuring hardware registers, controlling LEDs, reading sensors, setting up communication protocols. Let's master this together!

📚 TABLE OF CONTENTS

1. **Why Bits Matter in Embedded Systems**
2. **Number Systems Crash Course (Binary, Hex)**
3. **AND Operator (&)**
4. **OR Operator (|)**
5. **XOR Operator (^)**
6. **NOT Operator (~)**
7. **Left Shift (<<)**
8. **Right Shift (>>)**
9. **Bit Masks — The Master Skill**
10. **The 4 Holy Operations (SET, CLEAR, TOGGLE, CHECK)**
11. **Register Bit-Field Manipulation**
12. **Struct Bit-Fields in C**
13. **Portability Across Word Sizes**
14. **Real Embedded Project Examples**
15. **Common Mistakes & Traps**
16. **Cheat Sheet**

1️⃣ WHY BITS MATTER IN EMBEDDED SYSTEMS

In embedded systems, you're talking directly to hardware. Hardware doesn't understand "turn on the LED" — it understands **individual bits** in **registers**.

```text
Imagine a register is a row of 8 light switches on a wall:

  Switch:  7    6    5    4    3    2    1    0
           | OFF | ON  | OFF | OFF | ON  | OFF | ON  | OFF |
Binary:    0    1    0    0    1    0    1    0

Each switch controls something different:
  - Switch 0: Red LED
```

```
  - Switch 1: Green LED
  - Switch 2: Blue LED
  - Switch 3: Buzzer
  - Switch 4: Motor
  - Switch 5: Fan
  - Switch 6: Heater
  - Switch 7: Emergency Stop

YOUR JOB: Flip individual switches WITHOUT touching the others!
That's what bitwise operations do!
```

2️⃣ NUMBER SYSTEMS CRASH COURSE

Before we touch operators, you MUST be comfortable with binary and hex.

C

```
/*
 * ============================================================
 * 📐 BINARY (Base 2) — The language hardware speaks
 * ============================================================
 *
 * Each digit is a "bit" and can only be 0 or 1
 * Bits are numbered from RIGHT to LEFT, starting at 0
 *
 *   Bit position:  7   6   5   4   3   2   1   0
 *   Place value:  128  64  32  16   8   4   2   1
 *
 *   Example: What is 0b01001010 in decimal?
 *
 *   Bit 6 = 1 → 64
 *   Bit 3 = 1 → 8
 *   Bit 1 = 1 → 2
 *                ──
 *   Total:       74
 *
 * ============================================================
 * 🔷 HEXADECIMAL (Base 16) — Shorthand for binary
 * ============================================================
 *
 * Why hex? Because 1 hex digit = exactly 4 bits!
 * Much easier to read than long binary strings.
 *
 *   Hex:  0  1  2  3  4  5  6  7  8  9  A   B   C   D   E   F
 *   Dec:  0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15
 *   Bin: 0000 0001 0010 0011 0100 0101 0110 0111
 *        1000 1001 1010 1011 1100 1101 1110 1111
 *
 *   Example: 0x4A
 *     4 = 0100
 *     A = 1010
 *     → 0100 1010 = 74 decimal   (Same as our example above!)
 *
 *   ⚡ PRO TIP: In embedded systems, ALWAYS use hex (0x...)
 *   for bit operations. Never use decimal. It's unreadable!
 *
 *   Bad:  register = 74;      // What bits are set?? Who knows!
 *   Good: register = 0x4A;    // 0100 1010 — clear!
 *   Good: register = 0b01001010;  // Even clearer! (C23/GCC extension)
 */
```

```c
#include <stdio.h>

int main(void)
{
    /* All three lines store EXACTLY the same value */
    unsigned char decimal_way = 74;          /* Hard to see bits    */
    unsigned char hex_way     = 0x4A;        /* Easy to see nibbles */
    unsigned char binary_way  = 0b01001010;  /* Easiest (GCC/C23)  */

    printf("Decimal: %u\n", decimal_way);    /* 74 */
    printf("Hex:     0x%02X\n", hex_way);    /* 0x4A */
    printf("Binary way: %u\n", binary_way);  /* 74 */

    return 0;
}

/*
 * 🧠 MEMORIZE THESE POWER-OF-2 VALUES:
 *
 *   Bit 0 = 1       = 0x01
 *   Bit 1 = 2       = 0x02
 *   Bit 2 = 4       = 0x04
 *   Bit 3 = 8       = 0x08
 *   Bit 4 = 16      = 0x10
 *   Bit 5 = 32      = 0x20
 *   Bit 6 = 64      = 0x40
 *   Bit 7 = 128     = 0x80
 *
 *   These are the values of each individual bit.
 *   You'll use these CONSTANTLY!
 */
```

3️⃣ AND OPERATOR (&) — The Gatekeeper

C

```c
/*
 * ==============================================================
 * 🔒 BITWISE AND (&) — Both bits must be 1 to get 1
 * ==============================================================
 *
 * Truth Table:
 *   0 & 0 = 0
 *   0 & 1 = 0
 *   1 & 0 = 0
 *   1 & 1 = 1    ← Only case that produces 1!
 *
 * Think of AND as a STRICT DOORMAN:
 *   "Both of you must have tickets (1) to enter!"
 *
 * Visual Example:
 *
 *       1 0 1 1 0 1 1 0    (0xB6 = 182)
 *     & 1 1 0 0 1 1 0 0    (0xCC = 204)
 *     ─────────────────
 *       1 0 0 0 0 1 0 0    (0x84 = 132)
 *       ↑         ↑
 *   Both are 1   Both are 1
 *   → result 1   → result 1
```

```
 *
 * ⭐ PRIMARY USE: Checking if specific bits are set
 *                 Clearing specific bits (with inverted mask)
 */

#include <stdio.h>
#include <stdint.h>  /* Use this for uint8_t, uint16_t, etc. */

int main(void)
{
    uint8_t a = 0xB6;  /* 1011 0110 */
    uint8_t b = 0xCC;  /* 1100 1100 */

    uint8_t result = a & b;  /* 1000 0100 = 0x84 */
    printf("0x%02X & 0x%02X = 0x%02X\n", a, b, result);

    /* ─── REAL EMBEDDED USE CASE: Checking a bit ─── */
    /*
     * Scenario: You have a status register. Bit 3 tells you
     * if a sensor is ready. Is it ready?
     */
    uint8_t status_register = 0b01001010;  /* Some status byte */

    /*
     * To CHECK bit 3, AND with a mask that has ONLY bit 3 set:
     *
     *   0100 1010   (status register)
     * & 0000 1000   (mask = 0x08 = bit 3)
     * ───────────
     *   0000 1000   → Non-zero! Bit 3 IS set!
     *
     * If bit 3 were 0:
     *   0100 0010   (status register with bit 3 = 0)
     * & 0000 1000   (mask)
     * ───────────
     *   0000 0000   → Zero! Bit 3 is NOT set!
     */
    if (status_register & 0x08) {
        printf("Sensor is READY! (bit 3 is set)\n");
    } else {
        printf("Sensor NOT ready (bit 3 is clear)\n");
    }

    /* ─── ANOTHER USE: Extracting specific bits ─── */
    /*
     * Scenario: A temperature sensor packs data like this:
     *   Bits 7-4: Whole degrees (0-15)
     *   Bits 3-0: Fractional part
     *
     * Reading: 0b1010_0111 → Whole=10, Fraction=7
     */
    uint8_t temp_reading = 0xA7;  /* 1010 0111 */

    /* Extract lower 4 bits (fractional part) */
    uint8_t fraction = temp_reading & 0x0F;  /* 0000 1111 mask */
    /*
     *   1010 0111
     * & 0000 1111    ← This mask "lets through" only bits 0-3
     * ───────────
     *   0000 0111  = 7
     */

    /* Extract upper 4 bits (whole degrees) */
    uint8_t whole = (temp_reading & 0xF0) >> 4;  /* Shift down too */
```

```
    /*
     *   1010 0111
     * & 1111 0000    ← This mask "lets through" only bits 4-7
     * ───────────
     *   1010 0000  = 0xA0
     *   >> 4
     *   0000 1010  = 10
     */

    printf("Temperature: %u.%u degrees\n", whole, fraction);
    /* Output: Temperature: 10.7 degrees */

    return 0;
}

/*
 * 🧠 REMEMBER THIS RULE:
 *   AND with 1 → keeps the original bit (passes through)
 *   AND with 0 → always gives 0 (blocks/clears the bit)
 *
 *   Think of 1 as "transparent glass" and 0 as "brick wall"
 */
```

4️⃣ OR OPERATOR (|) — The Includer

C

```
/*
 * ═══════════════════════════════════════════════════════════
 * 🟢 BITWISE OR (|) — Either bit being 1 gives 1
 * ═══════════════════════════════════════════════════════════
 *
 * Truth Table:
 *   0 | 0 = 0    ← Only case that produces 0!
 *   0 | 1 = 1
 *   1 | 0 = 1
 *   1 | 1 = 1
 *
 * Think of OR as a GENEROUS HOST:
 *   "If EITHER of you has a ticket, you can both come in!"
 *
 * Visual Example:
 *
 *       1 0 1 0 0 0 0 0    (0xA0)
 *     | 0 0 0 0 0 1 0 1    (0x05)
 *     ─────────────────
 *       1 0 1 0 0 1 0 1    (0xA5)
 *
 * ⭐ PRIMARY USE: Setting specific bits to 1
 *                 (Without disturbing other bits!)
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── REAL EMBEDDED USE CASE: Setting bits ─── */
    /*
     * Scenario: You have a control register for GPIO pins.
```

```
     * Currently pins 5 and 7 are configured as outputs.
     * You want to ALSO make pin 2 an output.
     * YOU MUST NOT CHANGE PINS 5 AND 7!
     *
     *   Current register: 1010 0000  (pins 7,5 are outputs)
     *   You want:         1010 0100  (pins 7,5,2 are outputs)
     */
    uint8_t gpio_direction = 0xA0;  /* 1010 0000 — pins 7,5 set */

    /* Set bit 2 using OR */
    gpio_direction = gpio_direction | 0x04;   /* 0000 0100 */
    /*
     *   1010 0000   (original)
     * | 0000 0100   (mask with bit 2 set)
     * ──────────
     *   1010 0100   ✅ Bit 2 is now set! Bits 7,5 untouched!
     */

    printf("GPIO Direction: 0x%02X\n", gpio_direction);  /* 0xA4 */

    /* ─── Shorter syntax: the compound assignment operator ─── */
    /* These two lines do EXACTLY the same thing: */
    gpio_direction = gpio_direction | 0x04;   /* Long form  */
    gpio_direction |= 0x04;                   /* Short form (preferred!) */

    /* ─── Setting MULTIPLE bits at once ─── */
    /*
     * Scenario: Enable UART module.
     * Need to set bit 0 (enable) and bit 3 (TX enable) and bit 4 (RX enable)
     */
    uint8_t uart_control = 0x00;  /* All disabled */

    /* Method 1: One at a time (works but verbose) */
    uart_control |= 0x01;  /* Set bit 0: Enable */
    uart_control |= 0x08;  /* Set bit 3: TX enable */
    uart_control |= 0x10;  /* Set bit 4: RX enable */
    /* Result: 0001 1001 = 0x19 */

    /* Method 2: All at once (cleaner!) */
    uart_control = 0x00;   /* Reset for demo */
    uart_control |= (0x01 | 0x08 | 0x10);  /* Set bits 0, 3, 4 */
    /*
     *   0000 0001   (bit 0)
     * | 0000 1000   (bit 3)
     * | 0001 0000   (bit 4)
     * ──────────
     *   0001 1001   = 0x19  ✅ Same result, one operation!
     */

    printf("UART Control: 0x%02X\n", uart_control);  /* 0x19 */

    return 0;
}

/*
 * 🧠 REMEMBER THIS RULE:
 *   OR with 1 → always gives 1 (forces bit ON)
 *   OR with 0 → keeps the original bit (no change)
 *
 *   Think of OR as "painting" — you can paint new bits ON,
 *   but you can't paint them OFF.
 */
```

5️⃣ XOR OPERATOR (^) — The Toggler

C

```c
/*
 * ==============================================================
 * 🔄 BITWISE XOR (^) — Bits must be DIFFERENT to get 1
 * ==============================================================
 *
 * XOR = "eXclusive OR"
 *
 * Truth Table:
 *   0 ^ 0 = 0    ← Same → 0
 *   0 ^ 1 = 1    ← Different → 1
 *   1 ^ 0 = 1    ← Different → 1
 *   1 ^ 1 = 0    ← Same → 0
 *
 * Think of XOR as a DIFFERENCE DETECTOR:
 *   "Are you two different? Then yes (1)!"
 *
 * ⭐ PRIMARY USE: Toggling specific bits (flip 0→1 or 1→0)
 *
 * Visual:
 *       1 0 1 1 0 1 1 0    (0xB6)
 *     ^ 0 0 0 0 1 1 1 1    (0x0F — toggle lower 4 bits)
 *     ─────────────────
 *       1 0 1 1 1 0 0 1    (0xB9)
 *                 ↑ ↑ ↑ ↑
 *          These 4 bits got FLIPPED!
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── REAL EMBEDDED USE CASE 1: Toggle an LED ─── */
    /*
     * Scenario: An LED is connected to bit 5 of PORT B.
     * You want it to BLINK — toggle on/off repeatedly.
     *
     * You DON'T know the current state — and you don't care!
     * XOR will flip it regardless.
     */
    uint8_t PORTB = 0b01100010;  /* Some initial state */

    printf("Before toggle: 0x%02X\n", PORTB);

    PORTB ^= 0x20;  /* Toggle bit 5 (0010 0000) */
    /*
     *   0110 0010   (bit 5 was 1)
     * ^ 0010 0000
     * ──────────
     *   0100 0010   (bit 5 is now 0!)
     *      ↑
     *   Flipped!
     */
    printf("After toggle 1: 0x%02X\n", PORTB);

    PORTB ^= 0x20;  /* Toggle bit 5 again */
    /*
     *   0100 0010   (bit 5 was 0)
     * ^ 0010 0000
```

```
     * ──────────
     *   0110 0010   (bit 5 is now 1 again!)
     */
    printf("After toggle 2: 0x%02X\n", PORTB);
    /* It's back to the original! XOR is its own inverse! */

    /* ─── XOR MAGIC PROPERTY: Self-inverse ─── */
    /*
     * A ^ B ^ B = A   (XOR something twice = original!)
     *
     * This is used in:
     *   - Simple encryption
     *   - Error detection
     *   - Swapping variables without temp
     */

    /* ─── COOL TRICK: Swap without temporary variable ─── */
    uint8_t x = 0x3C;  /* 0011 1100 */
    uint8_t y = 0xA5;  /* 1010 0101 */

    printf("\nBefore swap: x=0x%02X, y=0x%02X\n", x, y);

    x ^= y;  /* x now holds x^y                          */
    y ^= x;  /* y = y ^ (x^y) = x  (original x!)         */
    x ^= y;  /* x = (x^y) ^ x = y  (original y!)         */

    printf("After swap:  x=0x%02X, y=0x%02X\n", x, y);
    /* x is now 0xA5, y is now 0x3C — swapped! */

    /* ─── USE CASE 2: Detect which bits changed ─── */
    /*
     * Scenario: You read a GPIO port twice. Which pins changed?
     */
    uint8_t old_reading = 0b11001010;
    uint8_t new_reading = 0b11010010;

    uint8_t changed_bits = old_reading ^ new_reading;
    /*
     *   1100 1010   (old)
     * ^ 1101 0010   (new)
     * ──────────
     *   0001 1000   Bits 3 and 4 changed!
     */
    printf("\nChanged pins: 0x%02X\n", changed_bits);  /* 0x18 */

    /* Check which specific pins changed */
    for (int i = 0; i < 8; i++) {
        if (changed_bits & (1 << i)) {
            printf("  Pin %d changed!\n", i);
        }
    }
    /* Output:
     *   Pin 3 changed!
     *   Pin 4 changed!
     */

    return 0;
}

/*
 * 🧠 REMEMBER THIS RULE:
 *   XOR with 1 → FLIPS the bit (toggles)
 *   XOR with 0 → keeps the original bit (no change)
 *
```

```
 *    ⚡ XOR is its own inverse: A ^ B ^ B = A
 */
```

6️⃣ NOT OPERATOR (~) — The Inverter

C

```c
/*
 * ============================================================
 * 🔃 BITWISE NOT (~) — Flips ALL bits (one's complement)
 * ============================================================
 *
 * This is a UNARY operator (takes only ONE operand)
 *
 * Truth Table:
 *   ~0 = 1
 *   ~1 = 0
 *
 * Visual:
 *   ~ 1 0 1 1 0 1 1 0    (0xB6)
 *   ─────────────────
 *     0 1 0 0 1 0 0 1    (0x49)
 *     Every single bit is flipped!
 *
 * ⚠️ WARNING: NOT is tricky because it depends on the TYPE SIZE!
 *    ~(uint8_t)0x0F  = 0xF0      (8-bit: flips to F0)
 *    ~(uint32_t)0x0F = 0xFFFFFF0  (32-bit: lots of F's!)
 *
 * ⭐ PRIMARY USE: Creating inverse masks for clearing bits
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── Basic NOT ─── */
    uint8_t a = 0xB6;   /* 1011 0110 */
    uint8_t b = ~a;      /* 0100 1001 = 0x49 */

    printf("~0x%02X = 0x%02X\n", a, b);  /* ~0xB6 = 0x49 */

    /* ─── THE MOST IMPORTANT USE: Clearing bits! ─── */
    /*
     * Scenario: You want to CLEAR bit 3 in a register.
     * You need to AND with a mask that has bit 3 = 0 and all others = 1
     *
     * Hard way:  mask = 0b11110111   (manually figure out all the 1s)
     * Easy way:  mask = ~(1 << 3)    (let the compiler do it!)
     *
     * Let's trace ~(1 << 3):
     *   1 << 3  = 0000 1000       (bit 3 set)
     *   ~       = 1111 0111       (bit 3 clear, all others set!)
     *
     * This is MUCH better because:
     * 1. It's readable: "NOT bit 3" → clear bit 3
     * 2. It works for any bit width automatically
     * 3. You don't have to count 1s and 0s manually
     */
    uint8_t control_reg = 0xFF;  /* All bits set: 1111 1111 */
```

```
    /* Clear bit 3 */
    control_reg &= ~(1 << 3);   /* AND with 1111 0111 */
    /*
     *   1111 1111    (original)
     * & 1111 0111    (~(1 << 3))
     * ───────────
     *   1111 0111    ✅ Bit 3 cleared! Others untouched!
     */

    printf("After clearing bit 3: 0x%02X\n", control_reg);  /* 0xF7 */

    /* Clear multiple bits at once */
    control_reg = 0xFF;
    control_reg &= ~(  (1 << 3) | (1 << 5) | (1 << 7)  );
    /*
     *   (1 << 3) = 0000 1000
     *   (1 << 5) = 0010 0000
     *   (1 << 7) = 1000 0000
     *   OR them:   1010 1000   (bits 3, 5, 7 set)
     *   NOT:       0101 0111   (bits 3, 5, 7 cleared)
     *
     *   1111 1111
     * & 0101 0111
     * ───────────
     *   0101 0111  = 0x57   ✅ Bits 3, 5, 7 all cleared!
     */
    printf("After clearing bits 3,5,7: 0x%02X\n", control_reg);

    /* ── ⚠️ THE TYPE SIZE TRAP ── */
    /*
     * NOT promotes to int! This can bite you!
     */
    uint8_t small = 0x0F;           /* 0000 1111 (8-bit) */
    unsigned int result = ~small;   /* NOT in 32-bit! */

    printf("\n~0x0F as uint8_t:  0x%02X\n", (uint8_t)~small);    /* 0xF0 ✅ */
    printf("~0x0F as uint32_t: 0x%08X\n", (uint32_t)~small);     /* 0xFFFFFF0 ⚠️
*/
    /* In 32-bit: 0x0F = 0x0000000F → ~0x0000000F = 0xFFFFFFF0 */
    /* That's a LOT of extra 1s you didn't expect! */

    return 0;
}

/*
 * 🧠 REMEMBER THIS RULE:
 *   ~ flips EVERY bit
 *   Use ~mask to create "inverse masks" for clearing bits
 *   ⚠️ Always be aware of the data type size!
 */
```

7️⃣ LEFT SHIFT (<<) — The Multiplier

C

```
/*
 * ═══════════════════════════════════════════════════════════
 * ⬅️ LEFT SHIFT (<<) — Shifts all bits to the LEFT
 * ═══════════════════════════════════════════════════════════
```

```
 *
 * What it does:
 *   - Every bit moves N positions to the LEFT
 *   - Empty positions on the RIGHT are filled with 0
 *   - Bits that fall off the LEFT are LOST
 *
 * Visual (shift left by 2):
 *
 *   Before: [0][0][1][0][1][1][0][1]   = 0x2D (45)
 *             ↑ ↑
 *   Lost! ←┘ └← Every bit moves 2 left
 *
 *   After:  [1][0][1][1][0][1][0][0]   = 0xB4 (180)
 *                               ↑   ↑
 *                         Filled with 0
 *
 * 💡 MATH SHORTCUT: Left shift by N = Multiply by 2^N
 *   45 << 2 = 45 × 4 = 180  ✅
 *
 * ⭐ PRIMARY USE #1: Creating bit masks — (1 << n) gives you bit n
 * ⭐ PRIMARY USE #2: Fast multiplication by powers of 2
 * ⭐ PRIMARY USE #3: Positioning values into bit fields
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── Basic left shift ─── */
    uint8_t a = 0x01;   /* 0000 0001 = 1 */

    printf("0x%02X << 0 = 0x%02X  (bit 0)\n", a, (uint8_t)(a << 0));  /* 0x01 */
    printf("0x%02X << 1 = 0x%02X  (bit 1)\n", a, (uint8_t)(a << 1));  /* 0x02 */
    printf("0x%02X << 2 = 0x%02X  (bit 2)\n", a, (uint8_t)(a << 2));  /* 0x04 */
    printf("0x%02X << 3 = 0x%02X  (bit 3)\n", a, (uint8_t)(a << 3));  /* 0x08 */
    printf("0x%02X << 4 = 0x%02X  (bit 4)\n", a, (uint8_t)(a << 4));  /* 0x10 */
    printf("0x%02X << 5 = 0x%02X  (bit 5)\n", a, (uint8_t)(a << 5));  /* 0x20 */
    printf("0x%02X << 6 = 0x%02X  (bit 6)\n", a, (uint8_t)(a << 6));  /* 0x40 */
    printf("0x%02X << 7 = 0x%02X  (bit 7)\n", a, (uint8_t)(a << 7));  /* 0x80 */

    /*
     * ⚡ SEE THE PATTERN?
     * (1 << n) gives you a number with ONLY bit n set!
     *
     *   (1 << 0) = 0000 0001 = 0x01 = 1
     *   (1 << 1) = 0000 0010 = 0x02 = 2
     *   (1 << 2) = 0000 0100 = 0x04 = 4
     *   (1 << 3) = 0000 1000 = 0x08 = 8
     *   (1 << 4) = 0001 0000 = 0x10 = 16
     *   (1 << 5) = 0010 0000 = 0x20 = 32
     *   (1 << 6) = 0100 0000 = 0x40 = 64
     *   (1 << 7) = 1000 0000 = 0x80 = 128
     *
     * THIS IS THE MOST IMPORTANT PATTERN IN EMBEDDED C!
     * (1 << n) is how you create bit masks!
     */

    /* ─── USE CASE: Named bits using defines ─── */
    /*
     * Real embedded code uses #define to name bits:
     */
    #define LED_RED     (1 << 0)   /* Bit 0 = 0x01 */
    #define LED_GREEN   (1 << 1)   /* Bit 1 = 0x02 */
```

```
    #define LED_BLUE    (1 << 2)   /* Bit 2 = 0x04 */
    #define BUZZER      (1 << 3)   /* Bit 3 = 0x08 */
    #define MOTOR       (1 << 4)   /* Bit 4 = 0x10 */

    uint8_t outputs = 0x00;

    /* Now code reads like ENGLISH! */
    outputs |= LED_RED;               /* Turn on red LED */
    outputs |= LED_GREEN | MOTOR;     /* Turn on green LED and motor */
    outputs &= ~BUZZER;               /* Turn off buzzer */

    printf("\nOutputs: 0x%02X\n", outputs);  /* 0x13 = red+green+motor */

    /* ─── USE CASE: Positioning a value into a bit field ─── */
    /*
     * Scenario: A timer register looks like this:
     *   Bits 7-5: Prescaler (0-7)
     *   Bits 4-2: Mode (0-7)
     *   Bits 1-0: Clock source (0-3)
     *
     * You want: Prescaler=5, Mode=3, Clock=2
     */
    uint8_t timer_reg = 0;

    timer_reg |= (5 << 5);   /* Prescaler: 101 shifted to bits 7-5 */
    timer_reg |= (3 << 2);   /* Mode: 011 shifted to bits 4-2      */
    timer_reg |= (2 << 0);   /* Clock: 10 shifted to bits 1-0      */
    /*
     *   Prescaler(5): 101..... = 1010 0000
     *   Mode(3):      ...011.. = 0000 1100
     *   Clock(2):     ......10 = 0000 0010
     *   OR together:             1010 1110 = 0xAE
     */
    printf("Timer register: 0x%02X\n", timer_reg);  /* 0xAE */

    return 0;
}
```

8️⃣ RIGHT SHIFT (>>) — The Divider

C

```
/*
 * ═══════════════════════════════════════════════════════════
 * ➡️ RIGHT SHIFT (>>) — Shifts all bits to the RIGHT
 * ═══════════════════════════════════════════════════════════
 *
 * What it does:
 *   - Every bit moves N positions to the RIGHT
 *   - Bits that fall off the RIGHT are LOST
 *   - For UNSIGNED: left side filled with 0 (logical shift)
 *   - For SIGNED: left side filled with sign bit (arithmetic shift)
 *     ⚠️ This is implementation-defined! Avoid shifting signed!
 *
 * Visual (shift right by 2):
 *
 *   Before: [1][0][1][1][0][1][0][0]   = 0xB4 (180)
 *                                 ↑   ↑
 *                   bits move right → Lost!
 *
```

```
 *   After:  [0][0][1][0][1][1][0][1]   = 0x2D (45)
 *             ↑   ↑
 *      Filled with 0 (unsigned)
 *
 * 💡 MATH SHORTCUT: Right shift by N = Divide by 2^N (integer division)
 *   180 >> 2 = 180 / 4 = 45  ✅
 *
 * ⭐ PRIMARY USE #1: Extracting values from bit fields
 * ⭐ PRIMARY USE #2: Fast division by powers of 2
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── Basic right shift ─── */
    uint8_t value = 0xB4;  /* 1011 0100 = 180 */

    printf("0x%02X >> 1 = 0x%02X  (%u)\n", value, value >> 1, value >> 1);  /*
90 */
    printf("0x%02X >> 2 = 0x%02X  (%u)\n", value, value >> 2, value >> 2);  /*
45 */
    printf("0x%02X >> 4 = 0x%02X  (%u)\n", value, value >> 4, value >> 4);  /*
11 */

    /* ─── USE CASE: Extract a bit field ─── */
    /*
     * Remember our timer register from before?
     *   Bits 7-5: Prescaler (0-7)
     *   Bits 4-2: Mode (0-7)
     *   Bits 1-0: Clock source (0-3)
     *
     * Register value: 0xAE = 1010 1110
     *                        ^^^ ^^^  ^^
     *                        Pre Mode Clock
     */
    uint8_t timer_reg = 0xAE;  /* 1010 1110 */

    /* Extract clock source (bits 1-0) */
    uint8_t clock_src = timer_reg & 0x03;  /* Mask lower 2 bits */
    /* 1010 1110 & 0000 0011 = 0000 0010 = 2 ✅ */

    /* Extract mode (bits 4-2) — need to shift down! */
    uint8_t mode = (timer_reg >> 2) & 0x07;  /* Shift down, then mask 3 bits */
    /*
     * Step 1: 1010 1110 >> 2 = 0010 1011
     * Step 2: 0010 1011 & 0000 0111 = 0000 0011 = 3 ✅
     */

    /* Extract prescaler (bits 7-5) */
    uint8_t prescaler = (timer_reg >> 5) & 0x07;
    /*
     * Step 1: 1010 1110 >> 5 = 0000 0101
     * Step 2: 0000 0101 & 0000 0111 = 0000 0101 = 5 ✅
     */

    printf("Timer Register: 0x%02X\n", timer_reg);
    printf("  Clock source: %u\n", clock_src);    /* 2 */
    printf("  Mode:         %u\n", mode);          /* 3 */
    printf("  Prescaler:    %u\n", prescaler);     /* 5 */

    /* ─── PATTERN: Extract any bit field ─── */
    /*
```

```
     * UNIVERSAL FORMULA to extract bits from position P with width W:
     *
     *   extracted_value = (register >> P) & ((1 << W) - 1)
     *
     * Where:
     *   P = starting bit position (lowest bit of the field)
     *   W = width (number of bits in the field)
     *   ((1 << W) - 1) creates a mask of W ones
     *
     * Example: Extract 3 bits starting at position 2:
     *   (register >> 2) & ((1 << 3) - 1)
     *   (register >> 2) & (8 - 1)
     *   (register >> 2) & 7
     *   (register >> 2) & 0x07
     *
     * Let's make a REUSABLE MACRO for this:
     */
    #define EXTRACT_BITS(reg, pos, width) \
        (((reg) >> (pos)) & ((1U << (width)) - 1U))

    /* Now extraction is a one-liner! */
    printf("\nUsing EXTRACT_BITS macro:\n");
    printf("  Clock:     %u\n", EXTRACT_BITS(timer_reg, 0, 2));  /* 2 */
    printf("  Mode:      %u\n", EXTRACT_BITS(timer_reg, 2, 3));  /* 3 */
    printf("  Prescaler: %u\n", EXTRACT_BITS(timer_reg, 5, 3));  /* 5 */

    /* ─── ⚠️ SIGNED RIGHT SHIFT WARNING ─── */
    /*
     * NEVER right-shift signed integers in embedded code!
     * The behavior is IMPLEMENTATION-DEFINED (not portable!)
     *
     * int8_t signed_val = -4;  // 1111 1100
     * signed_val >> 1;          // Could be 1111 1110 (-2) — arithmetic
     *                           // OR      0111 1110 (126) — logical
     *                           // Depends on compiler! DON'T DO THIS!
     *
     * ✅ ALWAYS use unsigned types for bit operations!
     */

    return 0;
}
```

9️⃣ BIT MASKS — THE MASTER SKILL

C

```
/*
 * ═══════════════════════════════════════════════════════════
 * 🎭 BIT MASKS — Your Most Important Tool
 * ═══════════════════════════════════════════════════════════
 *
 * A "mask" is a value designed to select specific bits.
 * Think of it like a stencil — it exposes the bits you care about
 * and blocks the rest.
 *
 *   Data:   1 0 1 1 0 1 1 0
 *   Mask:   0 0 1 1 1 1 0 0   ← "window" exposing bits 2-5
 *   ─────────────────────────
 *   AND:    0 0 1 1 0 1 0 0   ← Only bits 2-5 come through!
 *
```

```
 * There are several ways to BUILD masks:
 */

#include <stdio.h>
#include <stdint.h>

/* ─── HELPER: Print 8 bits for visualization ─── */
void print_bits(const char *label, uint8_t value)
{
    printf("%s 0x%02X = ", label, value);
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
        if (i == 4) printf(" ");  /* Space between nibbles */
    }
    printf("\n");
}

int main(void)
{
    printf("========== MASK CONSTRUCTION TECHNIQUES ==========\n\n");

    /* ─── Method 1: Single bit mask using (1 << n) ─── */
    /*
     * This is the MOST COMMON pattern in embedded C.
     * Creates a mask with exactly ONE bit set.
     */
    uint8_t mask_bit0 = (1 << 0);   /* 0000 0001 */
    uint8_t mask_bit3 = (1 << 3);   /* 0000 1000 */
    uint8_t mask_bit7 = (1 << 7);   /* 1000 0000 */

    print_bits("Bit 0 mask:", mask_bit0);
    print_bits("Bit 3 mask:", mask_bit3);
    print_bits("Bit 7 mask:", mask_bit7);

    /* ─── Method 2: Multi-bit mask using OR ─── */
    /*
     * Combine single-bit masks to select multiple bits.
     */
    uint8_t mask_bits_0_3_7 = (1 << 0) | (1 << 3) | (1 << 7);
    /* 0000 0001 | 0000 1000 | 1000 0000 = 1000 1001 */
    print_bits("\nBits 0,3,7:", mask_bits_0_3_7);

    /* ─── Method 3: Contiguous bit range mask ─── */
    /*
     * Need bits 2 through 5 set? (a range of consecutive bits)
     *
     * Formula: ((1 << width) - 1) << start_position
     *
     * Let's build a mask for bits 2-5 (4 bits wide, starting at 2):
     */
    uint8_t range_mask = ((1 << 4) - 1) << 2;
    /*
     * Step by step:
     *   (1 << 4)     = 0001 0000 = 16
     *   (16 - 1)     = 0000 1111 = 15   ← 4 consecutive 1s at bottom
     *   (15 << 2)    = 0011 1100        ← Shift up to start at bit 2
     *
     *   Result:        0011 1100  ✅ Bits 2,3,4,5 are set!
     */
    print_bits("\nBits 2-5:  ", range_mask);

    /* Let's make a MACRO for this common pattern */
    #define BITMASK_RANGE(start, width) \
        (((1U << (width)) - 1U) << (start))
```

```
print_bits("Bits 0-3:  ", BITMASK_RANGE(0, 4));  /* 0000 1111 */
print_bits("Bits 4-7:  ", BITMASK_RANGE(4, 4));  /* 1111 0000 */
print_bits("Bits 1-2:  ", BITMASK_RANGE(1, 2));  /* 0000 0110 */

/* ─── Method 4: Inverse mask using ~ ─── */
/*
 * Want everything EXCEPT certain bits? Use NOT!
 */
uint8_t all_except_bit3 = ~(1 << 3);
print_bits("\nAll except bit 3:", all_except_bit3);  /* 1111 0111 */

uint8_t all_except_2_to_5 = ~BITMASK_RANGE(2, 4);
print_bits("All except 2-5: ", all_except_2_to_5);  /* 1100 0011 */

/* ─── Method 5: Common pre-defined masks ─── */
uint8_t lower_nibble = 0x0F;   /* 0000 1111 — bits 0-3 */
uint8_t upper_nibble = 0xF0;   /* 1111 0000 — bits 4-7 */
uint8_t all_bits     = 0xFF;   /* 1111 1111 — all bits  */
uint8_t no_bits      = 0x00;   /* 0000 0000 — no bits   */

printf("\n========= REAL-WORLD MASK EXAMPLE =========\n\n");

/*
 * Scenario: ADC Configuration Register
 *
 *   Bit 7:    Enable (1=on, 0=off)
 *   Bits 6-4: Channel select (0-7)
 *   Bits 3-2: Resolution (0=8bit, 1=10bit, 2=12bit, 3=14bit)
 *   Bit 1:    Continuous mode (1=continuous, 0=single)
 *   Bit 0:    Start conversion (write 1 to start)
 *
 * Define named masks for everything:
 */
#define ADC_ENABLE          (1 << 7)
#define ADC_CHANNEL_MASK    BITMASK_RANGE(4, 3)    /* 0111 0000 */
#define ADC_CHANNEL_POS     4
#define ADC_RES_MASK        BITMASK_RANGE(2, 2)    /* 0000 1100 */
#define ADC_RES_POS         2
#define ADC_CONTINUOUS      (1 << 1)
#define ADC_START           (1 << 0)

/* Configure ADC: Enable, Channel 5, 12-bit, Continuous */
uint8_t adc_reg = 0;

adc_reg |= ADC_ENABLE;                    /* Set enable bit */
adc_reg |= (5 << ADC_CHANNEL_POS);        /* Channel 5 → bits 6-4 */
adc_reg |= (2 << ADC_RES_POS);            /* 12-bit (2) → bits 3-2 */
adc_reg |= ADC_CONTINUOUS;                /* Continuous mode */
adc_reg |= ADC_START;                     /* Start conversion */

print_bits("ADC config:", adc_reg);
/*
 * Expected: 1_101_10_1_1 = 1101 1011 = 0xDB
 *           E Ch5 12b C S
 */

/* Read back the channel */
uint8_t channel = (adc_reg & ADC_CHANNEL_MASK) >> ADC_CHANNEL_POS;
printf("ADC Channel: %u\n", channel);  /* 5 */

/* Change ONLY the channel to 3 (without touching other bits!) */
adc_reg &= ~ADC_CHANNEL_MASK;          /* Clear channel bits */
adc_reg |= (3 << ADC_CHANNEL_POS);     /* Set new channel */
```

```
    print_bits("New ADC:   ", adc_reg);
    channel = (adc_reg & ADC_CHANNEL_MASK) >> ADC_CHANNEL_POS;
    printf("New channel: %u\n", channel);  /* 3 */

    return 0;
}
```

🔟 THE 4 HOLY OPERATIONS (SET, CLEAR, TOGGLE, CHECK)

This is THE most important section. Memorize these 4 patterns!

C

```
/*
 * ============================================================
 * ⭐⭐⭐ THE 4 HOLY BIT OPERATIONS ⭐⭐⭐
 * ============================================================
 *
 * These are the 4 fundamental operations you'll use EVERY DAY.
 * They are the foundation of ALL register manipulation.
 *
 * Let's say we have a register REG and we want to manipulate bit N:
 *
 *
 *  ┌──────────────┬──────────────────────┬──────────────────────┐
 *  │ Operation    │ Code                 │ What it does         │
 *  ├──────────────┼──────────────────────┼──────────────────────┤
 *  │ SET          │ REG |=  (1 << N)     │ Force bit N to 1     │
 *  │ CLEAR        │ REG &= ~(1 << N)     │ Force bit N to 0     │
 *  │ TOGGLE       │ REG ^=  (1 << N)     │ Flip bit N           │
 *  │ CHECK        │ REG &   (1 << N)     │ Test if bit N is 1   │
 *  └──────────────┴──────────────────────┴──────────────────────┘
 *
 * That's it. Four patterns. Four. Memorize them and you can
 * control ANY hardware register on ANY microcontroller!
 */

#include <stdio.h>
#include <stdint.h>

/* ─── REUSABLE MACROS (Use these in your projects!) ─── */

/* Single bit operations */
#define BIT_SET(reg, bit)       ((reg) |=  (1U << (bit)))
#define BIT_CLEAR(reg, bit)     ((reg) &= ~(1U << (bit)))
#define BIT_TOGGLE(reg, bit)    ((reg) ^=  (1U << (bit)))
#define BIT_CHECK(reg, bit)     ((reg) &   (1U << (bit)))

/* Named-mask operations (when you have #defined masks) */
#define MASK_SET(reg, mask)     ((reg) |=  (mask))
#define MASK_CLEAR(reg, mask)   ((reg) &= ~(mask))
#define MASK_TOGGLE(reg, mask)  ((reg) ^=  (mask))
#define MASK_CHECK(reg, mask)   ((reg) &   (mask))

/* ─── Helper for visualization ─── */
void show_register(const char *label, uint8_t reg)
{
    printf("  %s 0x%02X = ", label, reg);
```

```
    for (int i = 7; i >= 0; i--) {
        printf("%c", (reg & (1 << i)) ? '1' : '0');
        if (i == 4) printf("_");
    }
    printf("\n");
}

int main(void)
{
    uint8_t reg = 0x00;  /* Start with all zeros */

    printf("===== DEMONSTRATING THE 4 HOLY OPERATIONS =====\n\n");

    /* ─── 1. SET: Turn a bit ON ─── */
    printf("── SET (|=) ──\n");
    show_register("Start:  ", reg);

    BIT_SET(reg, 0);    /* Set bit 0 */
    show_register("Set b0: ", reg);

    BIT_SET(reg, 4);    /* Set bit 4 */
    show_register("Set b4: ", reg);

    BIT_SET(reg, 7);    /* Set bit 7 */
    show_register("Set b7: ", reg);
    /* reg is now: 1001 0001 = 0x91 */

    /* ─── 2. CLEAR: Turn a bit OFF ─── */
    printf("\n── CLEAR (&= ~) ──\n");
    show_register("Start:  ", reg);

    BIT_CLEAR(reg, 4);  /* Clear bit 4 */
    show_register("Clr b4: ", reg);
    /* reg is now: 1000 0001 = 0x81 */

    /* ─── 3. TOGGLE: Flip a bit ─── */
    printf("\n── TOGGLE (^=) ──\n");
    show_register("Start:  ", reg);

    BIT_TOGGLE(reg, 0);  /* Flip bit 0 (was 1, becomes 0) */
    show_register("Tog b0: ", reg);

    BIT_TOGGLE(reg, 3);  /* Flip bit 3 (was 0, becomes 1) */
    show_register("Tog b3: ", reg);
    /* reg is now: 1000 1000 = 0x88 */

    /* ─── 4. CHECK: Test a bit ─── */
    printf("\n── CHECK (&) ──\n");
    show_register("Register:", reg);

    for (int i = 0; i < 8; i++) {
        if (BIT_CHECK(reg, i)) {
            printf("  Bit %d is SET (1)\n", i);
        } else {
            printf("  Bit %d is CLEAR (0)\n", i);
        }
    }

    printf("\n===== COMPLETE EMBEDDED EXAMPLE =====\n\n");

    /*
     * Real scenario: Controlling a traffic light system
     *
     * Register layout (8-bit GPIO output):
```

```
 *   Bit 0: North Red
 *   Bit 1: North Yellow
 *   Bit 2: North Green
 *   Bit 3: East Red
 *   Bit 4: East Yellow
 *   Bit 5: East Green
 *   Bit 6: Pedestrian Walk
 *   Bit 7: Emergency Flash
 */
#define NORTH_RED       (1 << 0)
#define NORTH_YELLOW    (1 << 1)
#define NORTH_GREEN     (1 << 2)
#define EAST_RED        (1 << 3)
#define EAST_YELLOW     (1 << 4)
#define EAST_GREEN      (1 << 5)
#define PED_WALK        (1 << 6)
#define EMERGENCY       (1 << 7)

/* Combine related masks */
#define NORTH_ALL   (NORTH_RED | NORTH_YELLOW | NORTH_GREEN)
#define EAST_ALL    (EAST_RED | EAST_YELLOW | EAST_GREEN)
#define ALL_LIGHTS  (NORTH_ALL | EAST_ALL)

uint8_t traffic_reg = 0x00;  /* All lights off */

/* State 1: North Green, East Red */
printf("State 1: North=GREEN, East=RED\n");
MASK_CLEAR(traffic_reg, ALL_LIGHTS);    /* Clear all lights first */
MASK_SET(traffic_reg, NORTH_GREEN);     /* North gets green */
MASK_SET(traffic_reg, EAST_RED);        /* East gets red */
show_register("Lights:", traffic_reg);

/* State 2: North Yellow, East Red */
printf("\nState 2: North=YELLOW, East=RED\n");
MASK_CLEAR(traffic_reg, NORTH_ALL);     /* Clear north lights */
MASK_SET(traffic_reg, NORTH_YELLOW);    /* North gets yellow */
/* East red stays on — we didn't touch it! */
show_register("Lights:", traffic_reg);

/* State 3: North Red, East Green */
printf("\nState 3: North=RED, East=GREEN\n");
MASK_CLEAR(traffic_reg, ALL_LIGHTS);    /* Clear everything */
MASK_SET(traffic_reg, NORTH_RED | EAST_GREEN);  /* Set both at once */
show_register("Lights:", traffic_reg);

/* Emergency! Flash everything! */
printf("\nEMERGENCY! Toggle emergency flash\n");
MASK_SET(traffic_reg, EMERGENCY);
show_register("Lights:", traffic_reg);

/* Check if emergency is active */
if (MASK_CHECK(traffic_reg, EMERGENCY)) {
    printf("⚠️  EMERGENCY MODE IS ACTIVE!\n");
}

/* Check which direction has green */
if (MASK_CHECK(traffic_reg, NORTH_GREEN)) {
    printf("North has green light\n");
}
if (MASK_CHECK(traffic_reg, EAST_GREEN)) {
    printf("East has green light\n");
}

return 0;
```

```
}
```

1️⃣1️⃣ REGISTER BIT-FIELD MANIPULATION (Real Hardware!)

C

```c
/*
 * ============================================================
 * 📟 REGISTER BIT-FIELD MANIPULATION
 * ============================================================
 *
 * In real microcontrollers (STM32, AVR, PIC, etc.), you access
 * hardware through MEMORY-MAPPED REGISTERS.
 *
 * These registers are at FIXED MEMORY ADDRESSES.
 * Each bit (or group of bits) controls something specific.
 *
 * Let's look at how this works from first principles.
 */

#include <stdio.h>
#include <stdint.h>

/*
 * ─── HOW HARDWARE REGISTERS WORK ───
 *
 * In a real microcontroller, registers are at fixed addresses:
 *
 *   Address 0x40020000 = GPIOA Mode Register
 *   Address 0x40020004 = GPIOA Output Type Register
 *   Address 0x40020014 = GPIOA Output Data Register
 *   ... etc.
 *
 * To access them in C, we use pointers:
 *
 *   #define GPIOA_MODER  (*(volatile uint32_t *)0x40020000)
 *
 * Let's break this down:
 *
 *   0x40020000                    ← The hardware address
 *   (uint32_t *)0x40020000       ← Cast address to pointer
 *   *(uint32_t *)0x40020000      ← Dereference: read/write the register
 *   volatile                      ← CRITICAL! Tells compiler:
 *                                    "This value can change at any time
 *                                     (by hardware), so NEVER optimize
 *                                     away reads/writes to it!"
 *
 * ⚠️ NEVER forget 'volatile' for hardware registers!
 *    Without it, the compiler might optimize away your writes,
 *    thinking they're unnecessary. Your hardware won't work!
 */

/* Simulated hardware register (for demo - can't use real addresses here) */
static volatile uint32_t simulated_register = 0;

/* Macro to access it (mimics real hardware access) */
#define MY_CONTROL_REG   simulated_register
```

```
/*
 * ─── COMPLETE EXAMPLE: UART Configuration ───
 *
 * Let's simulate configuring a UART (serial communication) module.
 *
 * UART Control Register 1 (32-bit):
 *
 *  Bit(s) │ Name       │ Description
 * ────────┼────────────┼──────────────────────────────
 *    0    │ UE         │ UART Enable (1=enable)
 *    1    │ RE         │ Receiver Enable
 *    2    │ TE         │ Transmitter Enable
 *   3-4   │ PS, PCE    │ Parity (00=none, 01=even, 10=odd)
 *   5-7   │ WAKE       │ Wakeup method
 *  8-11   │ Reserved   │ Do not modify!
 *  12-15  │ BRR        │ Baud rate divisor (0-15)
 *  16-31  │ Reserved   │ Do not modify!
 */

/* ── Step 1: Define bit positions ── */
#define UART_UE_POS         0       /* UART Enable position */
#define UART_RE_POS         1       /* Receiver Enable position */
#define UART_TE_POS         2       /* Transmitter Enable position */
#define UART_PARITY_POS     3       /* Parity start position */
#define UART_WAKE_POS       5       /* Wakeup start position */
#define UART_BRR_POS        12      /* Baud rate divisor position */

/* ── Step 2: Define masks ── */
#define UART_UE             (1U << UART_UE_POS)          /* 0x0001 */
#define UART_RE             (1U << UART_RE_POS)          /* 0x0002 */
#define UART_TE             (1U << UART_TE_POS)          /* 0x0004 */
#define UART_PARITY_MASK    (0x3U << UART_PARITY_POS)    /* 0x0018 */
#define UART_WAKE_MASK      (0x7U << UART_WAKE_POS)      /* 0x00E0 */
#define UART_BRR_MASK       (0xFU << UART_BRR_POS)       /* 0xF000 */

/*
 * 💡 EXPLANATION OF MASK CONSTRUCTION:
 *
 * UART_PARITY_MASK: 2-bit field starting at position 3
 *   0x3 = 0b11 (2 bits of all-ones)
 *   0x3 << 3 = 0b11000 = 0x18
 *   This creates: 0000 0000 0001 1000
 *                              ^^
 *                         bits 3-4
 *
 * UART_BRR_MASK: 4-bit field starting at position 12
 *   0xF = 0b1111 (4 bits of all-ones)
 *   0xF << 12 = 0xF000
 *   This creates: 1111 0000 0000 0000
 *                 ^^^^
 *              bits 12-15
 */

/* ── Step 3: Define field values ── */
#define UART_PARITY_NONE    (0U << UART_PARITY_POS)
#define UART_PARITY_EVEN    (1U << UART_PARITY_POS)
#define UART_PARITY_ODD     (2U << UART_PARITY_POS)

/* ── Step 4: Helper macros for multi-bit fields ── */
/*
 * To SET a multi-bit field:
 *   1. Clear the old value (AND with inverted mask)
 *   2. Set the new value (OR with shifted value)
 *
```

```
 * This is called "Read-Modify-Write" — the most common
 * pattern in embedded register programming!
 */
#define SET_FIELD(reg, mask, value) \
    do { \
        (reg) = ((reg) & ~(mask)) | ((value) & (mask)); \
    } while (0)

/*
 * 💡 WHY do { ... } while(0)?
 *
 * This is a C trick to make multi-statement macros safe.
 * It ensures the macro works correctly in all contexts:
 *
 *   if (condition)
 *       SET_FIELD(reg, mask, value);  // Safe! Acts as one statement
 *   else
 *       something_else();
 *
 * Without do-while(0), the if/else would break with a bare { }.
 */

#define GET_FIELD(reg, mask, pos) \
    (((reg) & (mask)) >> (pos))

void print_register(const char *label, uint32_t reg)
{
    printf("%s 0x%08X = ", label, reg);
    for (int i = 15; i >= 0; i--) {
        printf("%d", (reg >> i) & 1);
        if (i % 4 == 0 && i > 0) printf("_");
    }
    printf(" (showing bits 15-0)\n");
}

int main(void)
{
    printf("====== UART REGISTER CONFIGURATION ======\n\n");

    /* Start with register cleared */
    MY_CONTROL_REG = 0x00000000;
    print_register("Initial:   ", MY_CONTROL_REG);

    /* ── Configure UART step by step ── */

    /* 1. Set baud rate divisor to 9 */
    printf("\n1. Setting baud rate divisor to 9...\n");
    SET_FIELD(MY_CONTROL_REG, UART_BRR_MASK, (9U << UART_BRR_POS));
    print_register("After BRR: ", MY_CONTROL_REG);
    /*
     * What happened:
     *   reg & ~UART_BRR_MASK → Clears bits 12-15
     *   | (9 << 12) & UART_BRR_MASK → Sets 1001 at bits 12-15
     *
     *   0000_0000_0000_0000  (was)
     *   1001_0000_0000_0000  (now) — 9 in bits 12-15
     */

    /* 2. Set parity to Even */
    printf("\n2. Setting parity to EVEN...\n");
    SET_FIELD(MY_CONTROL_REG, UART_PARITY_MASK, UART_PARITY_EVEN);
    print_register("After PAR: ", MY_CONTROL_REG);

    /* 3. Enable UART, Transmitter, and Receiver */
```

```
    printf("\n3. Enabling UART, TX, and RX...\n");
    MY_CONTROL_REG |= UART_UE | UART_TE | UART_RE;
    print_register("After EN:  ", MY_CONTROL_REG);

    /* ── Read back configuration ── */
    printf("\n====== READ BACK CONFIGURATION ======\n");

    uint32_t baud_div = GET_FIELD(MY_CONTROL_REG, UART_BRR_MASK, UART_BRR_POS);
    uint32_t parity   = GET_FIELD(MY_CONTROL_REG, UART_PARITY_MASK,
UART_PARITY_POS);

    printf("UART Enabled:  %s\n", (MY_CONTROL_REG & UART_UE) ? "YES" : "NO");
    printf("TX Enabled:    %s\n", (MY_CONTROL_REG & UART_TE) ? "YES" : "NO");
    printf("RX Enabled:    %s\n", (MY_CONTROL_REG & UART_RE) ? "YES" : "NO");
    printf("Baud Divisor:  %u\n", baud_div);
    printf("Parity:        %s\n", parity == 0 ? "None" :
                                  parity == 1 ? "Even" : "Odd");

    /* ── Modify just the baud rate (without touching anything else!) ── */
    printf("\n====== CHANGE BAUD RATE TO 13 ======\n");
    printf("Before:\n");
    print_register("  REG:     ", MY_CONTROL_REG);

    SET_FIELD(MY_CONTROL_REG, UART_BRR_MASK, (13U << UART_BRR_POS));

    printf("After:\n");
    print_register("  REG:     ", MY_CONTROL_REG);
    printf("New baud divisor: %u\n",
           GET_FIELD(MY_CONTROL_REG, UART_BRR_MASK, UART_BRR_POS));

    /* Verify nothing else changed */
    printf("UART still enabled: %s\n",
           (MY_CONTROL_REG & UART_UE) ? "YES" : "NO");
    printf("Parity still even:  %s\n",
           GET_FIELD(MY_CONTROL_REG, UART_PARITY_MASK, UART_PARITY_POS) == 1
           ? "YES" : "NO");

    return 0;
}
```

1️⃣2️⃣ STRUCT BIT-FIELDS IN C

C

```
/*
 * ==============================================================
 * 📦 C STRUCT BIT-FIELDS — Another Way to Access Bits
 * ==============================================================
 *
 * C lets you define structs where fields are measured in BITS:
 *
 *   struct {
 *       unsigned int enable : 1;   // 1 bit wide
 *       unsigned int mode   : 3;   // 3 bits wide
 *   };
 *
 * This LOOKS super clean, but has SERIOUS portability problems
 * in embedded systems. Let me show you both sides.
 */
```

```c
#include <stdio.h>
#include <stdint.h>

/* ─── Bit-field struct for our UART example ─── */
typedef struct {
    uint32_t UE      : 1;   /* Bit 0:    UART Enable          */
    uint32_t RE      : 1;   /* Bit 1:    Receiver Enable       */
    uint32_t TE      : 1;   /* Bit 2:    Transmitter Enable    */
    uint32_t PARITY  : 2;   /* Bits 3-4: Parity setting        */
    uint32_t WAKE    : 3;   /* Bits 5-7: Wakeup method         */
    uint32_t RSVD1   : 4;   /* Bits 8-11: Reserved             */
    uint32_t BRR     : 4;   /* Bits 12-15: Baud rate divisor   */
    uint32_t RSVD2   : 16;  /* Bits 16-31: Reserved            */
} UART_Control_t;

/*
 * ⚠️ THE PROBLEMS WITH BIT-FIELDS:
 *
 * 1. BIT ORDER IS NOT GUARANTEED!
 *    - The C standard does NOT define whether bits are packed
 *      from LSB→MSB or MSB→LSB.
 *    - Different compilers may lay out fields differently!
 *    - GCC on ARM: usually LSB first (matches our layout above)
 *    - Other compilers: might be MSB first!
 *
 * 2. PADDING IS NOT GUARANTEED!
 *    - The compiler may insert padding between fields.
 *
 * 3. ATOMICITY IS NOT GUARANTEED!
 *    - Writing one field might require a read-modify-write
 *      of the entire word, which isn't atomic.
 *    - This can cause bugs with interrupts!
 *
 * 4. YOU CAN'T TAKE THE ADDRESS OF A BIT-FIELD!
 *    - &my_struct.UE  ← COMPILE ERROR!
 *
 * 🏆 VERDICT:
 *    - Bit-fields are OK for compiler-specific, same-platform code
 *    - For PORTABLE code, use manual bit manipulation (masks)
 *    - Many professional embedded codebases use BOTH:
 *      - union with a bitfield struct AND a raw uint32_t
 */

/* ─── THE UNION TRICK: Best of Both Worlds ─── */
typedef union {
    /* Access as individual fields (convenient but non-portable) */
    struct {
        uint32_t UE      : 1;
        uint32_t RE      : 1;
        uint32_t TE      : 1;
        uint32_t PARITY  : 2;
        uint32_t WAKE    : 3;
        uint32_t RSVD1   : 4;
        uint32_t BRR     : 4;
        uint32_t RSVD2   : 16;
    } bits;

    /* Access as raw 32-bit value (portable, for bit manipulation) */
    uint32_t raw;

} UART_Reg_t;

int main(void)
{
```

```
    UART_Reg_t uart_ctrl;
    uart_ctrl.raw = 0;  /* Clear everything */

    printf("====== USING BIT-FIELD STRUCT ======\n\n");

    /* Set fields using the struct (clean syntax!) */
    uart_ctrl.bits.UE     = 1;    /* Enable UART */
    uart_ctrl.bits.TE     = 1;    /* Enable transmitter */
    uart_ctrl.bits.RE     = 1;    /* Enable receiver */
    uart_ctrl.bits.PARITY = 1;    /* Even parity */
    uart_ctrl.bits.BRR    = 9;    /* Baud divisor = 9 */

    printf("Via bit-fields:\n");
    printf("  UE=%u, RE=%u, TE=%u, PARITY=%u, BRR=%u\n",
           uart_ctrl.bits.UE, uart_ctrl.bits.RE,
           uart_ctrl.bits.TE, uart_ctrl.bits.PARITY,
           uart_ctrl.bits.BRR);
    printf("  Raw value: 0x%08X\n\n", uart_ctrl.raw);

    /* Access the same data as raw bits */
    printf("Via raw register:\n");
    printf("  Bit 0 (UE):  %u\n", (uart_ctrl.raw >> 0) & 1);
    printf("  Bit 2 (TE):  %u\n", (uart_ctrl.raw >> 2) & 1);
    printf("  Bits 12-15:  %u\n", (uart_ctrl.raw >> 12) & 0xF);

    /* Mix both approaches */
    printf("\n====== MIXING APPROACHES ======\n\n");

    /* Set via raw (when you need guaranteed bit layout) */
    uart_ctrl.raw &= ~(0xFU << 12);    /* Clear BRR field */
    uart_ctrl.raw |=  (13U << 12);      /* Set BRR to 13 */

    /* Read via struct (convenient) */
    printf("BRR after raw modification: %u\n", uart_ctrl.bits.BRR);

    /*
     * 🏆 RECOMMENDATION FOR BEGINNERS:
     *
     * Start with MANUAL BIT MANIPULATION (masks and shifts).
     * It's:
     *   ✅ 100% portable
     *   ✅ You understand exactly what's happening
     *   ✅ Works identically on every compiler
     *   ✅ Industry standard approach
     *
     * Use bit-field structs only when:
     *   - You're on a single, known platform
     *   - Your team/vendor already uses them
     *   - You've verified the compiler's bit ordering
     */

    return 0;
}
```

1️⃣3️⃣ PORTABILITY ACROSS DIFFERENT WORD SIZES

C

```
/*
 * ============================================================
```

```
 * 🌍 PORTABILITY CONSIDERATIONS
 * ==============================================================
 *
 * Embedded systems use many different processors:
 *   - 8-bit:   AVR (Arduino), PIC, 8051
 *   - 16-bit:  MSP430, PIC24
 *   - 32-bit:  ARM Cortex-M (STM32, nRF52), ESP32
 *   - 64-bit:  ARM Cortex-A (Raspberry Pi), x86
 *
 * Your C code must handle the differences!
 *
 * THE BIG ISSUES:
 *   1. 'int' size varies (16-bit on AVR, 32-bit on ARM!)
 *   2. Integer promotion rules can surprise you
 *   3. Signed vs unsigned behavior differs
 *   4. Endianness (byte order) varies
 *   5. Bit-field layout varies
 */

#include <stdio.h>
#include <stdint.h>   /* ← YOUR BEST FRIEND! */
#include <limits.h>

int main(void)
{
    printf("====== YOUR PLATFORM INFO ======\n\n");

    printf("sizeof(char):      %zu bytes (%zu bits)\n", sizeof(char),
sizeof(char) * 8);
    printf("sizeof(short):     %zu bytes (%zu bits)\n", sizeof(short),
sizeof(short) * 8);
    printf("sizeof(int):       %zu bytes (%zu bits)\n", sizeof(int), sizeof(int)
* 8);
    printf("sizeof(long):      %zu bytes (%zu bits)\n", sizeof(long),
sizeof(long) * 8);
    printf("sizeof(long long): %zu bytes (%zu bits)\n", sizeof(long long),
sizeof(long long) * 8);

    /*
     * On your PC (64-bit), you'll likely see:
     *   char=1, short=2, int=4, long=8, long long=8
     *
     * On AVR (8-bit Arduino):
     *   char=1, short=2, int=2(!), long=4, long long=8
     *                         ↑
     *              int is only 16 bits!
     *
     * On ARM Cortex-M (STM32):
     *   char=1, short=2, int=4, long=4, long long=8
     */

    printf("\n====== PROBLEM #1: int SIZE VARIES ======\n\n");

    /*
     * THE PROBLEM:
     *   On AVR:  int = 16 bits → max value = 65535 (unsigned)
     *   On ARM:  int = 32 bits → max value = 4294967295
     *
     *   If you write:
     *     int x = 70000;
     *
     *   On ARM: x = 70000 ✅
     *   On AVR: x = 4464  ❌ (overflow! 70000 wraps around)
     */
```

```
/*
 * ✅ THE SOLUTION: ALWAYS use fixed-width types from <stdint.h>!
 *
 *   uint8_t   → Exactly 8 bits, unsigned  (0 to 255)
 *   uint16_t  → Exactly 16 bits, unsigned (0 to 65535)
 *   uint32_t  → Exactly 32 bits, unsigned (0 to 4294967295)
 *   uint64_t  → Exactly 64 bits, unsigned
 *
 *   int8_t    → Exactly 8 bits, signed    (-128 to 127)
 *   int16_t   → Exactly 16 bits, signed   (-32768 to 32767)
 *   int32_t   → Exactly 32 bits, signed
 *   int64_t   → Exactly 64 bits, signed
 *
 * ⚡ RULE: In embedded systems, NEVER use plain 'int'.
 *          ALWAYS use uint8_t, uint16_t, uint32_t, etc.
 */

/* Bad (non-portable): */
unsigned int bad_variable = 0xFFFF;  /* What size is this?? */

/* Good (portable, explicit): */
uint16_t good_variable = 0xFFFF;     /* Exactly 16 bits, always! */

printf("====== PROBLEM #2: INTEGER PROMOTION ======\n\n");

/*
 * THE PROBLEM:
 * C automatically "promotes" small types to 'int' in expressions!
 *
 * This means uint8_t and uint16_t get converted to int before
 * any arithmetic or bitwise operation is performed.
 *
 * This can cause UNEXPECTED RESULTS with NOT (~) and shifts!
 */

/* TRAP EXAMPLE 1: NOT on uint8_t */
uint8_t mask8 = 0x0F;          /* 0000 1111 */
/* uint8_t result = ~mask8;     What do you expect? 0xF0? */

/*
 * What ACTUALLY happens:
 *
 *   1. mask8 (uint8_t 0x0F) is PROMOTED to int
 *      → On 32-bit system: 0x0000000F
 *
 *   2. ~ is applied to the int:
 *      ~0x0000000F = 0xFFFFFFF0
 *
 *   3. Stored back to uint8_t:
 *      0xFFFFFFF0 truncated to 0xF0  ← Happens to work for uint8_t!
 *
 *   BUT if you store to uint32_t:
 *      uint32_t big_result = ~mask8;
 *      big_result = 0xFFFFFFF0  ← NOT 0x000000F0!
 *
 *   This is a common bug source!
 */

uint32_t trap_result = ~mask8;
printf("~(uint8_t)0x0F stored as uint32_t: 0x%08X\n", trap_result);
printf("  You might expect 0x000000F0, but got 0x%08X!\n\n", trap_result);

/* ✅ THE FIX: Cast after NOT, or use the correct width */
```

```
uint32_t safe_result = (uint8_t)~mask8;
printf("(uint8_t)~mask8 stored as uint32_t: 0x%08X ✅\n\n", safe_result);

/* TRAP EXAMPLE 2: Left shift overflow */
/*
 * On AVR (16-bit int):
 *   1 << 15 = -32768 (signed overflow! Undefined behavior!)
 *
 * On ARM (32-bit int):
 *   1 << 15 = 32768 (fine)
 *
 * ALWAYS use 1U (unsigned) and cast to correct size:
 */

/* Bad (potentially undefined on 16-bit platforms): */
/* int shifted = 1 << 15; */

/* Good: */
uint16_t shifted_safe = (uint16_t)1U << 15;   /* 0x8000, always */
uint32_t shifted_32   = 1UL << 31;             /* 0x80000000, always */
/*
 * 💡 SUFFIX GUIDE:
 *   1    → int (size varies!)
 *   1U   → unsigned int (size varies, but at least no sign issues)
 *   1UL  → unsigned long (at least 32 bits)
 *   1ULL → unsigned long long (at least 64 bits)
 */

printf("====== PROBLEM #3: SAFE SHIFT MACROS ======\n\n");

/*
 * Here are PORTABLE macros that work on ALL platforms:
 */

/* Portable single-bit mask macros */
#define BIT8(n)   ((uint8_t)1  << (n))   /* For 8-bit registers  */
#define BIT16(n)  ((uint16_t)1 << (n))   /* For 16-bit registers */
#define BIT32(n)  (1UL << (n))            /* For 32-bit registers */

/* Portable bit operations */
#define SET_BIT8(reg, n)    ((reg) |=  BIT8(n))
#define CLR_BIT8(reg, n)    ((reg) &= (uint8_t)~BIT8(n))
#define TGL_BIT8(reg, n)    ((reg) ^=  BIT8(n))
#define GET_BIT8(reg, n)    (((reg) >> (n)) & 1U)

#define SET_BIT32(reg, n)   ((reg) |=  BIT32(n))
#define CLR_BIT32(reg, n)   ((reg) &= ~BIT32(n))
#define TGL_BIT32(reg, n)   ((reg) ^=  BIT32(n))
#define GET_BIT32(reg, n)   (((reg) >> (n)) & 1UL)

/* Demo */
uint8_t reg8 = 0;
SET_BIT8(reg8, 3);
SET_BIT8(reg8, 7);
printf("8-bit register:  0x%02X\n", reg8);     /* 0x88 */

uint32_t reg32 = 0;
SET_BIT32(reg32, 3);
SET_BIT32(reg32, 31);
printf("32-bit register: 0x%08X\n", reg32);     /* 0x80000008 */

printf("\n====== PROBLEM #4: ENDIANNESS ======\n\n");

/*
```

```
 * ENDIANNESS: The order bytes are stored in memory.
 *
 * Value: 0x12345678
 *
 * LITTLE-ENDIAN (ARM Cortex-M, x86, most modern):
 *   Address:  0x00  0x01  0x02  0x03
 *   Value:    0x78  0x56  0x34  0x12
 *             ↑ Least significant byte first
 *
 * BIG-ENDIAN (some PowerPC, network byte order):
 *   Address:  0x00  0x01  0x02  0x03
 *   Value:    0x12  0x34  0x56  0x78
 *             ↑ Most significant byte first
 *
 * 💡 For BITWISE operations: Endianness doesn't matter!
 *    Bitwise ops work on the VALUE, not the memory layout.
 *    0xFF & 0x0F = 0x0F regardless of endianness.
 *
 * ⚠️ Endianness DOES matter when:
 *    - Reading/writing multi-byte data to hardware
 *    - Network communication
 *    - Accessing memory through byte pointers
 *    - Sharing data between different architectures
 */

/* Detect endianness at runtime */
uint32_t test_value = 0x01020304;
uint8_t *bytes = (uint8_t *)&test_value;

if (bytes[0] == 0x04) {
    printf("This system is LITTLE-ENDIAN\n");
    printf("  0x01020304 stored as: %02X %02X %02X %02X\n",
           bytes[0], bytes[1], bytes[2], bytes[3]);
} else {
    printf("This system is BIG-ENDIAN\n");
    printf("  0x01020304 stored as: %02X %02X %02X %02X\n",
           bytes[0], bytes[1], bytes[2], bytes[3]);
}

printf("\n====== GOLDEN RULES FOR PORTABILITY ======\n\n");

/*
 * 🏆 THE 10 GOLDEN RULES:
 *
 * 1. ALWAYS use <stdint.h> types (uint8_t, uint32_t, etc.)
 *    Never use plain int, short, long for hardware.
 *
 * 2. ALWAYS use unsigned types for bit operations.
 *    Signed right-shift is implementation-defined!
 *
 * 3. ALWAYS use 1U (or 1UL) instead of 1 in shifts.
 *    (1 << 31) is undefined behavior on 32-bit int!
 *    (1UL << 31) is perfectly defined.
 *
 * 4. Cast ~ results to the appropriate width.
 *    ~(uint8_t)x promotes to int first!
 *
 * 5. Never shift by more than the width of the type.
 *    (uint8_t)1 << 8 is UNDEFINED BEHAVIOR!
 *
 * 6. Never shift by a negative amount.
 *    That's undefined behavior too.
 *
 * 7. Use volatile for all hardware register pointers.
```

```
     *
     * 8. Don't rely on bit-field layout order.
     *    It's implementation-defined.
     *
     * 9. Be careful with endianness when sharing data
     *    between processors or over networks.
     *
     * 10. Document your register layouts clearly!
     *     Future you will thank present you.
     */

    return 0;
}
```

1️⃣4️⃣ REAL EMBEDDED PROJECT EXAMPLES

C

```
/*
 * ============================================================
 * 🔥 REAL-WORLD EMBEDDED EXAMPLES
 * ============================================================
 *
 * These are patterns you'll see in actual embedded firmware.
 */

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

/* ─── EXAMPLE 1: LED Pattern Controller ─── */
/*
 * 8 LEDs connected to a GPIO port.
 * Create various lighting patterns using bitwise ops.
 */
void led_pattern_demo(void)
{
    uint8_t leds = 0;

    printf("\n=== LED Pattern Controller ===\n");

    /* Pattern 1: Knight Rider (one LED bouncing back and forth) */
    printf("Knight Rider Pattern:\n");
    for (int i = 0; i < 8; i++) {
        leds = (1 << i);  /* Only one bit set, moves left */
        printf("  LEDs: 0x%02X  ", leds);
        for (int b = 7; b >= 0; b--)
            printf("%c", (leds & (1 << b)) ? '*' : '.');
        printf("\n");
    }
    for (int i = 6; i > 0; i--) {
        leds = (1 << i);  /* Now moves back right */
        printf("  LEDs: 0x%02X  ", leds);
        for (int b = 7; b >= 0; b--)
            printf("%c", (leds & (1 << b)) ? '*' : '.');
        printf("\n");
    }

    /* Pattern 2: Fill from left */
    printf("\nFill from left:\n");
```

```
    leds = 0;
    for (int i = 7; i >= 0; i--) {
        leds |= (1 << i);  /* SET one more bit each time */
        printf("  LEDs: 0x%02X  ", leds);
        for (int b = 7; b >= 0; b--)
            printf("%c", (leds & (1 << b)) ? '*' : '.');
        printf("\n");
    }

    /* Pattern 3: Alternating (checkerboard) */
    printf("\nAlternating toggle:\n");
    leds = 0xAA;  /* 1010 1010 */
    for (int i = 0; i < 6; i++) {
        printf("  LEDs: 0x%02X  ", leds);
        for (int b = 7; b >= 0; b--)
            printf("%c", (leds & (1 << b)) ? '*' : '.');
        printf("\n");
        leds ^= 0xFF;  /* Toggle ALL bits — flips pattern! */
    }
}

/* ─── EXAMPLE 2: Keyboard Matrix Scanner ─── */
/*
 * 4x4 keyboard matrix.
 * Rows are outputs (bits 3-0), Columns are inputs (bits 7-4).
 * To scan: drive one row LOW, read which columns go LOW.
 */
void keyboard_demo(void)
{
    printf("\n=== Keyboard Matrix Scanner ===\n");

    /* Simulated keymap: which key is at [row][col] */
    const char keymap[4][4] = {
        {'1', '2', '3', 'A'},
        {'4', '5', '6', 'B'},
        {'7', '8', '9', 'C'},
        {'*', '0', '#', 'D'},
    };

    /* Simulated: someone is pressing '5' (row 1, col 1) */
    uint8_t simulated_columns[4] = {
        0xF0,   /* Row 0: no keys pressed (all columns high) */
        0xD0,   /* Row 1: column 1 is low (bit 5 = 0) → '5' */
        0xF0,   /* Row 2: no keys pressed */
        0xF0,   /* Row 3: no keys pressed */
    };

    /* Scan all rows */
    for (int row = 0; row < 4; row++) {
        /*
         * In real hardware:
         *   GPIO_OUT &= ~(1 << row);  // Drive this row LOW
         *   columns = GPIO_IN & 0xF0;  // Read column pins
         *   GPIO_OUT |= (1 << row);    // Release row
         */
        uint8_t columns = simulated_columns[row];

        /* Check each column (bits 4-7) */
        for (int col = 0; col < 4; col++) {
            uint8_t col_bit = 4 + col;  /* Column pins are bits 4-7 */

            /* A pressed key pulls the column LOW (0) */
            if (!(columns & (1 << col_bit))) {
                printf("Key pressed: '%c' (Row %d, Col %d)\n",
```

```
                        keymap[row][col], row, col);
            }
        }
    }
}

/* ─── EXAMPLE 3: Status Flags System ─── */
/*
 * Use bits as compact boolean flags.
 * Instead of 8 separate bool variables (8 bytes),
 * store 8 flags in a single byte (1 byte)!
 * This is common in embedded systems with limited RAM.
 */
typedef uint8_t SystemFlags;

#define FLAG_POWER_ON       (1U << 0)   /* System powered up          */
#define FLAG_SENSOR_READY   (1U << 1)   /* Sensor initialized         */
#define FLAG_DATA_AVAILABLE (1U << 2)   /* New data ready to read     */
#define FLAG_ERROR          (1U << 3)   /* Error condition detected    */
#define FLAG_BUSY           (1U << 4)   /* System is processing       */
#define FLAG_CONNECTED      (1U << 5)   /* Communication link active  */
#define FLAG_LOW_BATTERY    (1U << 6)   /* Battery below threshold    */
#define FLAG_ALARM          (1U << 7)   /* Alarm condition active     */

void print_flags(SystemFlags flags)
{
    printf("System Status [0x%02X]: ", flags);
    if (flags & FLAG_POWER_ON)       printf("PWR ");
    if (flags & FLAG_SENSOR_READY)   printf("SNS ");
    if (flags & FLAG_DATA_AVAILABLE) printf("DAT ");
    if (flags & FLAG_ERROR)          printf("ERR ");
    if (flags & FLAG_BUSY)           printf("BSY ");
    if (flags & FLAG_CONNECTED)      printf("CON ");
    if (flags & FLAG_LOW_BATTERY)    printf("LOW ");
    if (flags & FLAG_ALARM)          printf("ALM ");
    if (flags == 0)                  printf("(none)");
    printf("\n");
}

void status_flags_demo(void)
{
    printf("\n=== Status Flags System ===\n");

    SystemFlags status = 0;    /* Everything off */
    print_flags(status);

    /* Boot sequence */
    printf("\n-- Boot sequence --\n");
    status |= FLAG_POWER_ON;          /* Power up */
    print_flags(status);

    status |= FLAG_SENSOR_READY;      /* Sensor initialized */
    print_flags(status);

    status |= FLAG_CONNECTED;         /* Link established */
    print_flags(status);

    /* Normal operation */
    printf("\n-- Normal operation --\n");
    status |= FLAG_DATA_AVAILABLE;    /* New data arrived */
    print_flags(status);

    /* Process the data */
    status |= FLAG_BUSY;              /* Start processing */
```

```
    status &= ~FLAG_DATA_AVAILABLE;   /* Consumed the data */
    print_flags(status);

    status &= ~FLAG_BUSY;             /* Done processing */
    print_flags(status);

    /* Error! */
    printf("\n-- Error condition --\n");
    status |= FLAG_ERROR | FLAG_ALARM;
    print_flags(status);

    /* Clear error */
    status &= ~(FLAG_ERROR | FLAG_ALARM);  /* Clear both at once */
    print_flags(status);

    /* Check multiple flags at once */
    printf("\n-- Complex checks --\n");

    /* Is the system fully operational? (power + sensor + connected, no error)
*/
    SystemFlags required = FLAG_POWER_ON | FLAG_SENSOR_READY | FLAG_CONNECTED;
    if ((status & required) == required && !(status & FLAG_ERROR)) {
        printf("✅ System fully operational!\n");
    }
    /*
     * 💡 WHY (status & required) == required?
     *
     * Just (status & required) checks if ANY of the bits are set.
     * == required checks that ALL of them are set!
     *
     *   status   = 0010 0011   (PWR + SNS + CON)
     *   required = 0010 0011
     *   status & required = 0010 0011
     *   == required? YES! ✅
     *
     *   status   = 0000 0001   (only PWR)
     *   required = 0010 0011
     *   status & required = 0000 0001
     *   == required? NO! ❌ (SNS and CON are missing)
     */

    /* Count how many flags are set (popcount) */
    int count = 0;
    SystemFlags temp = status;
    while (temp) {
        count += temp & 1;   /* Check lowest bit */
        temp >>= 1;          /* Shift right to next bit */
    }
    printf("Active flags: %d out of 8\n", count);
}

/* ─── EXAMPLE 4: Simple Circular Buffer with Bitmask Index ─── */
/*
 * 💡 COOL TRICK: If your buffer size is a power of 2,
 * you can use a bitmask instead of modulo (%) for wrapping!
 * Bitmask is MUCH faster than modulo on microcontrollers.
 *
 *   index = (index + 1) % 256;     // Slow (division!)
 *   index = (index + 1) & 0xFF;    // Fast (single AND instruction!)
 *
 * This works because:
 *   Powers of 2 in binary are: 100, 1000, 10000, etc.
 *   Minus 1 gives:             011, 0111, 01111, etc. ← a mask!
 *   AND with this mask does modulo for free!
```

```
 */

#define BUFFER_SIZE     16          /* MUST be power of 2! */
#define BUFFER_MASK     (BUFFER_SIZE - 1)   /* 0x0F = 0000 1111 */

typedef struct {
    uint8_t data[BUFFER_SIZE];
    uint8_t head;   /* Write index */
    uint8_t tail;   /* Read index */
} CircularBuffer;

void buf_init(CircularBuffer *buf)
{
    buf->head = 0;
    buf->tail = 0;
}

void buf_write(CircularBuffer *buf, uint8_t value)
{
    buf->data[buf->head] = value;
    buf->head = (buf->head + 1) & BUFFER_MASK;  /* Fast wrap! */
    /*
     * If head was 15 (0x0F):
     *   15 + 1 = 16 = 0001 0000
     *   & 0x0F     = 0000 0000  → Wrapped back to 0!
     *
     * Same as % 16, but uses a single AND instruction
     * instead of expensive division hardware!
     */
}

uint8_t buf_read(CircularBuffer *buf)
{
    uint8_t value = buf->data[buf->tail];
    buf->tail = (buf->tail + 1) & BUFFER_MASK;  /* Fast wrap! */
    return value;
}

bool buf_is_empty(CircularBuffer *buf)
{
    return buf->head == buf->tail;
}

uint8_t buf_count(CircularBuffer *buf)
{
    return (buf->head - buf->tail) & BUFFER_MASK;  /* Works too! */
}

void circular_buffer_demo(void)
{
    printf("\n=== Circular Buffer (Power-of-2 Trick) ===\n");

    CircularBuffer buf;
    buf_init(&buf);

    /* Write some values */
    for (uint8_t i = 10; i < 18; i++) {
        buf_write(&buf, i);
        printf("Wrote %u (head=%u)\n", i, buf.head);
    }

    printf("Buffer count: %u\n", buf_count(&buf));

    /* Read them back */
```

```
    while (!buf_is_empty(&buf)) {
        printf("Read: %u (tail=%u)\n", buf_read(&buf), buf.tail);
    }
}

/* ─── MAIN ─── */
int main(void)
{
    led_pattern_demo();
    keyboard_demo();
    status_flags_demo();
    circular_buffer_demo();

    return 0;
}
```

1️⃣5️⃣ COMMON MISTAKES & TRAPS

C

```
/*
 * ═══════════════════════════════════════════════════════════
 * ⚠️ COMMON MISTAKES — Learn from others' pain!
 * ═══════════════════════════════════════════════════════════
 */

#include <stdio.h>
#include <stdint.h>

int main(void)
{
    /* ─── MISTAKE 1: Confusing & with && ─── */
    /*
     *   &  = BITWISE AND  (operates on individual bits)
     *   && = LOGICAL AND   (operates on true/false values)
     *
     *   These are COMPLETELY DIFFERENT!
     */
    uint8_t a = 0x06;  /* 0000 0110 */
    uint8_t b = 0x03;  /* 0000 0011 */

    printf("Bitwise AND: 0x%02X & 0x%02X = 0x%02X\n",
           a, b, a & b);    /* 0x02 (0000 0010) */
    printf("Logical AND: 0x%02X && 0x%02X = %d\n",
           a, b, a && b);   /* 1 (both non-zero = true) */
    /*
     * Same applies to | vs ||, and ~ vs !
     *   |  vs ||  (bitwise OR vs logical OR)
     *   ~  vs !   (bitwise NOT vs logical NOT)
     */

    /* ─── MISTAKE 2: Forgetting operator precedence ─── */
    /*
     * ⚠️ DANGEROUS: Bitwise operators have LOWER precedence
     * than comparison operators!
     *
     * This is a classic source of bugs!
     */
    uint8_t reg = 0x0F;
```

```
/* WRONG — this checks (reg) & (0x04 == 0x04), not (reg & 0x04) == 0x04 */
/* if (reg & 0x04 == 0x04) { ... }  ← BUG! */

/* CORRECT — always use parentheses with bitwise operators! */
if ((reg & 0x04) == 0x04) {
    printf("Bit 2 is set ✅\n");
}

/*
 * ⚡ PRECEDENCE TABLE (from highest to lowest):
 *
 *   ~          (bitwise NOT — unary, highest)
 *   << >>      (shifts)
 *   &          (AND)
 *   ^          (XOR)
 *   |          (OR)
 *
 * But ALL of these are BELOW  ==, !=, <, >, <=, >=
 *
 * 🏆 GOLDEN RULE: When in doubt, add parentheses!
 *    It costs nothing and prevents bugs.
 */

/* ─── MISTAKE 3: Shifting by the type width or more ─── */
/*
 * Shifting by >= the width of the type is UNDEFINED BEHAVIOR!
 */
uint8_t val8 = 1;
/* uint8_t bad = val8 << 8;  ← UNDEFINED! 8-bit shifted by 8! */
/* The result could be ANYTHING! */

/* ✅ Make sure shift amount < type width */
uint32_t val32 = 1;
uint32_t ok = val32 << 31;  /* Fine: 31 < 32 */
/* uint32_t bad = val32 << 32;  ← UNDEFINED BEHAVIOR! */

printf("1 << 31 = 0x%08X\n", ok);

/* ─── MISTAKE 4: Signed integer right shift ─── */
int8_t signed_val = -4;   /* 1111 1100 in two's complement */
int8_t shifted = signed_val >> 1;
/*
 * What is the result?
 *   Arithmetic shift: 1111 1110 = -2 (sign bit preserved)
 *   Logical shift:    0111 1110 = 126 (zero-filled)
 *
 * THE C STANDARD SAYS: Implementation-defined!
 * You literally don't know what you'll get!
 *
 * ✅ Always use unsigned types for bit operations.
 */
printf("Signed -4 >> 1 = %d (implementation-defined!)\n", shifted);

/* ─── MISTAKE 5: The 1 vs 1U problem ─── */
/*
 * The literal '1' is of type 'int'.
 * On a 16-bit platform: int = 16 bits.
 * So (1 << 15) on such a platform overflows signed int
 * → UNDEFINED BEHAVIOR!
 */

/* Bad on 16-bit platforms: */
/* int x = 1 << 15;   ← UB: signed overflow! */
```

```
    /* Good everywhere: */
    uint16_t x16 = 1U << 15;     /* Unsigned, fine */
    uint32_t x32 = 1UL << 31;    /* Unsigned long, fine */
    printf("1U << 15 = 0x%04X\n", x16);    /* 0x8000 */
    printf("1UL << 31 = 0x%08X\n", x32);   /* 0x80000000 */

    /* ─── MISTAKE 6: Modifying reserved bits ─── */
    /*
     * Many hardware registers have "reserved" bits that must
     * not be modified! The datasheet says "write 0" or
     * "write as read" for these bits.
     *
     * WRONG:
     *   MY_REG = 0x05;  // Writes 0 to ALL other bits, including reserved!
     *
     * CORRECT (Read-Modify-Write):
     *   MY_REG |= 0x05;  // Only sets bits 0 and 2, preserves others
     *
     * Or to change specific bits:
     *   uint32_t temp = MY_REG;       // Read
     *   temp &= ~(bits_to_clear);     // Modify (clear target bits)
     *   temp |= new_value;            // Modify (set new value)
     *   MY_REG = temp;                // Write
     */

    printf("\n⚡ Remember: Parentheses are free. Bugs are expensive.\n");

    return 0;
}
```

1️⃣6️⃣ THE ULTIMATE CHEAT SHEET

text

```
╔═══════════════════════════════════════════════════════════════╗
╠═══               BITWISE OPERATIONS CHEAT SHEET              ═══╣
╠═══               For Embedded Systems in C                   ═══╣
╠═══════════════════════════════════════════════════════════════╣
╠═                                                               ═╣
╠═  ── OPERATORS ──                                              ═╣
╠═  &   AND     Both must be 1       0b1100 & 0b1010 = 0b1000   ═╣
╠═  |   OR      Either can be 1      0b1100 | 0b1010 = 0b1110   ═╣
╠═  ^   XOR     Must be different     0b1100 ^ 0b1010 = 0b0110   ═╣
╠═  ~   NOT     Flip all bits         ~0b1100 = 0b0011 (8-bit)   ═╣
╠═  <<  LSHIFT  Move bits left        0b0001 << 3 = 0b1000       ═╣
╠═  >>  RSHIFT  Move bits right       0b1000 >> 3 = 0b0001       ═╣
╠═                                                               ═╣
╠═  ── THE 4 HOLY OPERATIONS ──                                  ═╣
╠═  SET bit N:     reg |=  (1U << N)                             ═╣
╠═  CLEAR bit N:   reg &= ~(1U << N)                             ═╣
╠═  TOGGLE bit N:  reg ^=  (1U << N)                             ═╣
╠═  CHECK bit N:   if (reg & (1U << N))                          ═╣
╠═                                                               ═╣
╠═  ── MULTI-BIT FIELD OPERATIONS ──                             ═╣
╠═  Create mask:   ((1U << width) - 1U) << position              ═╣
╠═  Read field:    (reg & mask) >> position                      ═╣
╠═  Write field:   reg = (reg & ~mask) | (value << pos)          ═╣
╠═                                                               ═╣
╠═  ── PORTABILITY RULES ──                                      ═╣
╠═  ✅ Use uint8_t, uint16_t, uint32_t (never plain int)         ═╣
```

```
✅ Use 1U or 1UL in shifts (never plain 1)
✅ Use unsigned types for all bit operations
✅ Always use volatile for hardware registers
✅ Always add parentheses around bitwise expressions
✅ Never shift by >= type width
✅ Use Read-Modify-Write for register changes

── QUICK REFERENCE ──
(1 << 0) = 0x01 = 1        (1 << 4) = 0x10 = 16
(1 << 1) = 0x02 = 2        (1 << 5) = 0x20 = 32
(1 << 2) = 0x04 = 4        (1 << 6) = 0x40 = 64
(1 << 3) = 0x08 = 8        (1 << 7) = 0x80 = 128

── HEX ↔ BINARY ──
0=0000 4=0100 8=1000 C=1100
1=0001 5=0101 9=1001 D=1101
2=0010 6=0110 A=1010 E=1110
3=0011 7=0111 B=1011 F=1111

── COOL TRICKS ──
Check power of 2:  (n & (n-1)) == 0
Fast modulo (2^n): x & (n - 1)
Swap without temp:  a^=b; b^=a; a^=b;
Turn off lowest 1:  n & (n - 1)
Isolate lowest 1:   n & (-n)
Check if any set:   if (flags & mask)
Check if ALL set:   if ((flags & mask) == mask)
```

🗺️ WHERE TO GO FROM HERE

You've now learned **everything** you need about bitwise operations for embedded systems! Here's your practice path:

1. ✅ **Type out every example** — don't copy-paste. Muscle memory matters!
2. ✅ **Write a `print_bits()` function** from scratch to visualize any value
3. ✅ **Practice the 4 Holy Operations** until they're second nature
4. ✅ **Pick a real MCU datasheet** (like STM32F103) and try reading register descriptions — you'll understand them now!
5. ✅ **Build a status flag system** for a project of your own
6. ✅ **Write portable macros** using the patterns shown above

Remember these three golden rules:

> 🥇 **Always use unsigned fixed-width types** (`uint8_t`, `uint32_t`) 🥈 **Always use parentheses** around bitwise expressions 🥉 **Always use `1U` or `1UL`** in shift operations

The four holy operations — **SET, CLEAR, TOGGLE, CHECK** — are your bread and butter. Everything else builds on top of them. Master those four patterns and you can configure ANY hardware register on ANY microcontroller! 🚀